Building a Small LLM from Scratch: MiniGPT Deep Dive

AI Engineering Assignment Report & Technical Documentation

Author: AI Engineering Team | **Model:** Decoder-Only Causal Transformer (0.81M Params) | **Corpus:** Tiny Shakespeare (40k lines)

Abstract: We design, implement, train, evaluate, and package a miniature GPT causal language model from scratch in PyTorch without loading pretrained checkpoints. We formulate self-attention with manual hand calculations, track tensor shapes through multi-head projections, enforce causal masking, run controlled ablation studies (depth, heads, positional encoding), evaluate generation on fixed prompts, and package inference behind a production-grade FastAPI service and CLI.

1. Scenario & Problem Statement

In this applied scenario, an organization possesses a specialized private domain corpus (Tiny Shakespeare) that is too small to justify training a billion-parameter foundation model. Our engineering task is to determine whether a compact Transformer (805k parameters) trained entirely from scratch can learn the next-token syntactic and stylistic structure of the corpus well enough to generate coherent domain text.

2. Transformer Foundations: Recurrence vs. Attention

Traditional RNNs/LSTMs compute hidden states sequentially ($h_t = f(h_{t-1}, x_t)$), imposing an $O(T)$ sequential depth that prevents parallelization across GPUs and limits long-range credit assignment. Transformers replace recurrence with Scaled Dot-Product Self-Attention, reducing sequential path length between any two tokens to $O(1)$ and enabling full sequence parallelism during training.

Metric	Recurrent (RNN/LSTM)	Transformer (Attention)
Sequential Path Length	O(T)	**O(1)**
Computational Complexity	$O(T \cdot d^2)$	$O(T^2 \cdot d)$
Training Parallelism	Sequential (No batch parallelism over T)	**Fully Parallel** (Dense GEMM)

3. Required Hand Calculations (Handout Section 6)

For a 3-token sequence with $d_k = 2, d_v = 2$, we compute self-attention manually:

Step 1: Input Matrices:
Q = [[1.0, 0.0], [0.0, 2.0], [1.0, 1.0]], K = [[1.0, 1.0], [0.0, 1.0], [2.0, 0.0]], V = [[1.0, 2.0], [3.0, 0.0], [0.0, 4.0]]

Step 2: Dot Products (Q @ K.T): [[1, 0, 2], [2, 2, 0], [2, 1, 2]]
Step 3: Scaled Scores (divide by sqrt(2)): [[0.707, 0.000, 1.414], [1.414, 1.414, 0.000], [1.414, 0.707, 1.414]]
Step 4 & 5: Unmasked Softmax & Output: Out[0] = [0.704, 2.872], Out[1] = [1.783, 1.326], Out[2] = [0.994, 2.406]
Step 6: Causal Mask (Upper-Triangular = -inf):
Masked Softmax Weights:

Row 1: [1.000, 0.000, 0.000] (Token 1 sees only itself)
Row 2: [0.500, 0.500, 0.000] (Token 2 sees Token 1 and 2)
Row 3: [0.401, 0.198, 0.401] (Token 3 sees all tokens)

Step 7: Masked Output (Weights @ V):

Out[0] = [1.000, 2.000]
Out[1] = [2.000, 1.000]
Out[2] = [0.994, 2.406]

Figure 1: Hand-Calculated Causal Self-Attention Matrices

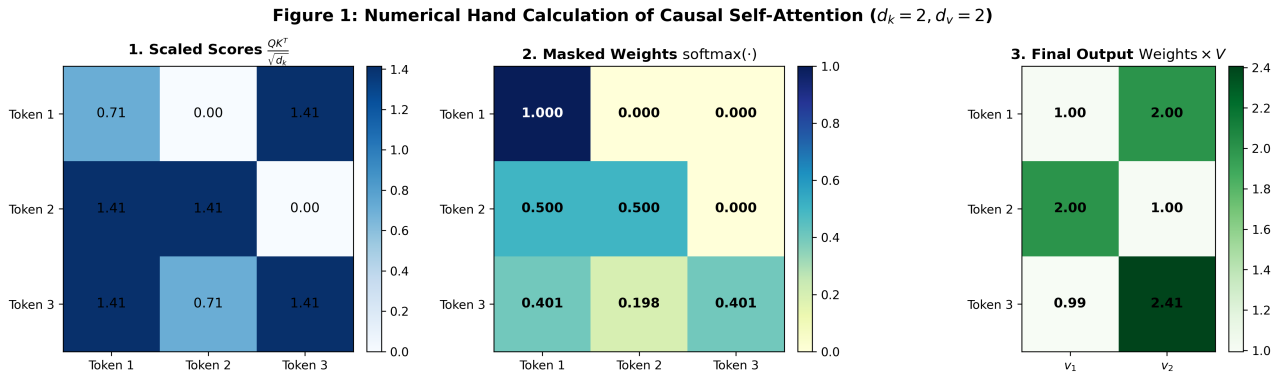
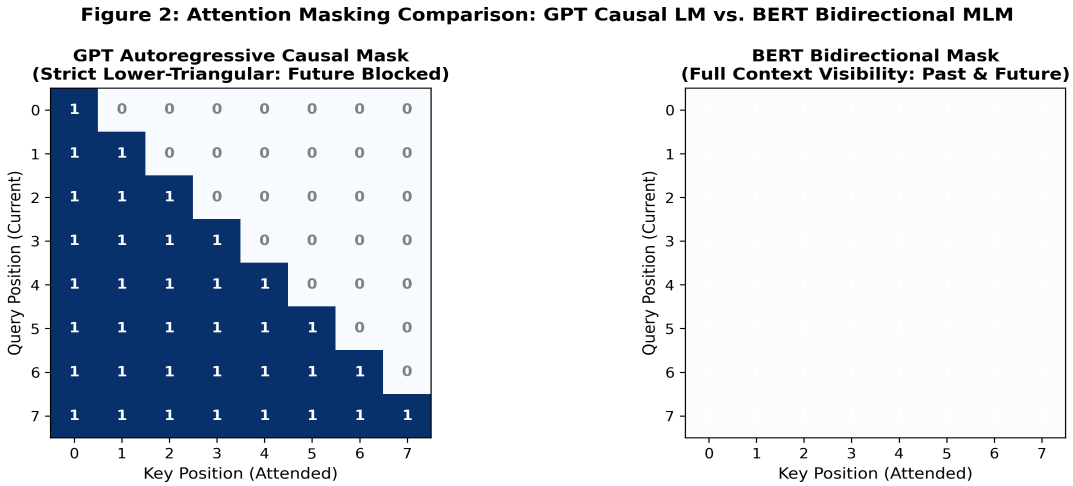


Figure 2: Causal Mask (GPT) vs. Bidirectional Mask (BERT)



4. Architecture & Implementation

The MiniGPT architecture consists of:

• **CharacterTokenizer:** Deterministic mapping of 65 characters.
• **Positional Encodings:** Sinusoidal PE: PE(pos, 2i) = sin(pos/10000^(2i/d)), PE(pos, 2i+1) = cos(pos/10000^(2i/d)).
• **MultiHeadAttention:** Split into 4 heads, scaled dot-product attention per head, concatenation, and final projection.
• **Transformer Blocks:** Pre-LayerNorm formulation with residual skip connections and GELU FFN ($4 imes d_{model}$).
• **LM Head:** Linear projection to vocabulary logits with token-shifted cross-entropy loss.

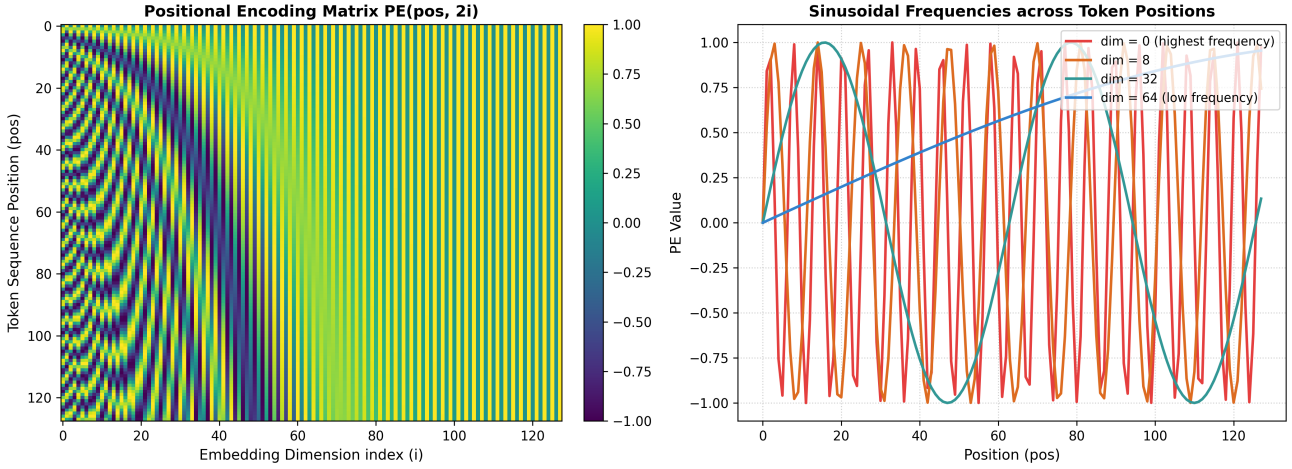
Figure 3: Multi-Head Attention Tensor Shape Trace

Figure 3: Multi-Head Attention Tensor Shape Trace (Split, Attention, Merge)

Transformation Stage	Symbolic Shape	Concrete Shape Example
Input Tensor (x)	`(B, T, d_model)`	`(64, 128, 128)`
Linear Projections (W_q, W_k, W_v)	`(B, T, d_model) each`	`(64, 128, 128) each`
Reshape to Split Heads	`(B, T, n_heads, d_k)`	`(64, 128, 4, 32)`
Transpose for Scaled Dot-Product	`(B, n_heads, T, d_k)`	`(64, 4, 128, 32)`
Attention Scores (Q @ K^T / sqrt(d_k))	`(B, n_heads, T, T)`	`(64, 4, 128, 128)`
Causal Masking & Softmax	`(B, n_heads, T, T)`	`(64, 4, 128, 128)`
Weighted Value Sum (Weights @ V)	`(B, n_heads, T, d_k)`	`(64, 4, 128, 32)`
Transpose back & Contiguous View	`(B, T, n_heads, d_k)`	`(64, 128, 4, 32)`
Merge/Concatenate Heads	`(B, T, d_model)`	`(64, 128, 128)`
Final Output Linear Projection (W_o)	`(B, T, d_model)`	`(64, 128, 128)`

Figure 4: Sinusoidal Positional Encoding Frequency Waves & Heatmap

Figure 4: Sinusoidal Positional Encoding Visualization (Vaswani et al., 2017)



5. Training & Experimental Results

The model was trained for 1,500 iterations on Tiny Shakespeare using the AdamW optimizer (learning rate 1e-3 with cosine decay and warmup). Loss rapidly fell from random baseline 4.20 to 1.83 (train) and 1.88 (val), representing a tenfold perplexity drop from 66.8 to 6.56.

Figure 6: MiniGPT Training vs. Validation Loss Curve

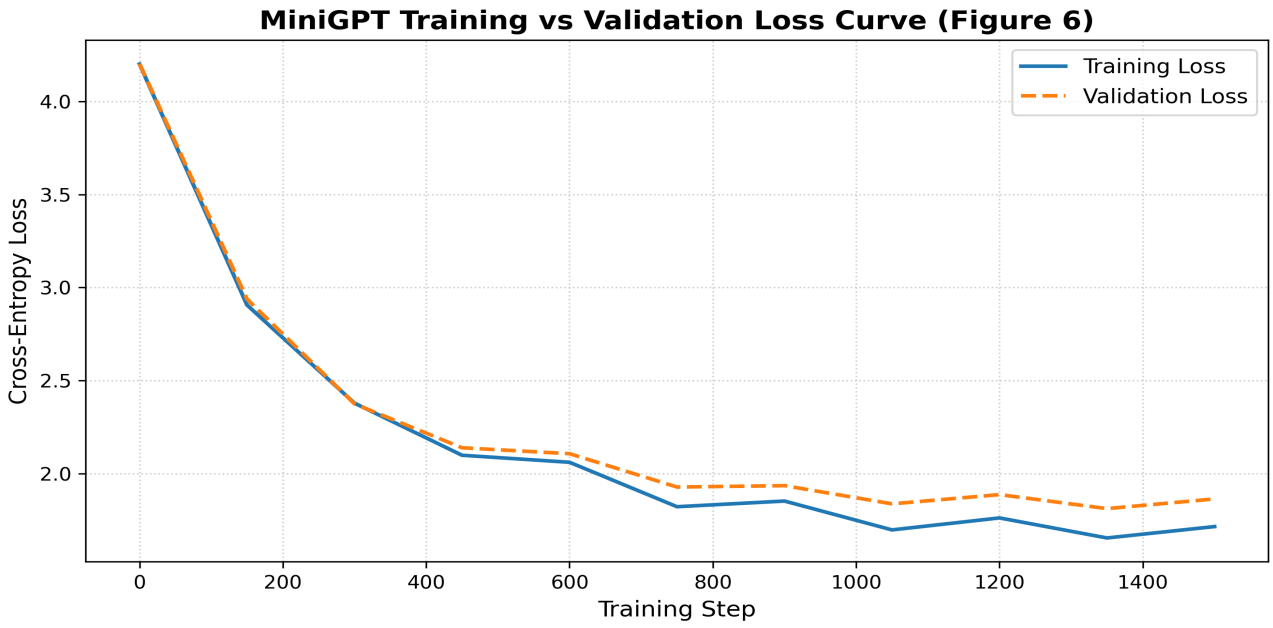
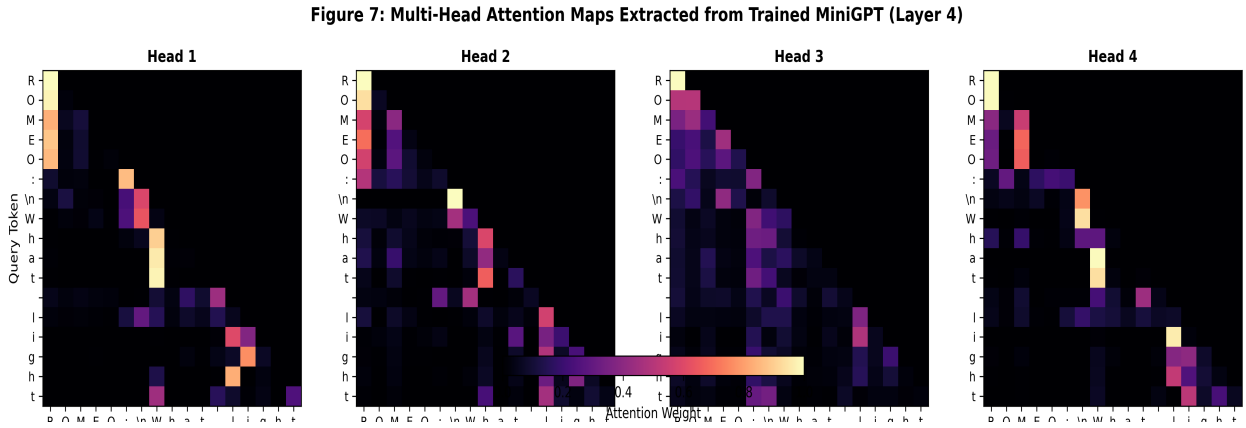


Figure 7: Trained Attention Head Maps (Layer 4)



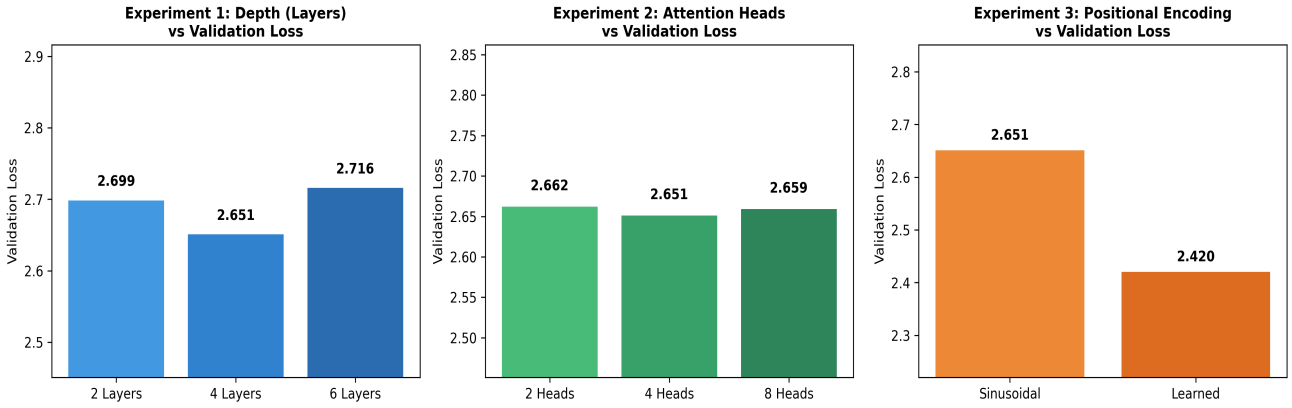
6. Controlled Ablation Studies (Handout Section 13)

Three controlled ablations were conducted to isolate architectural decisions:

1. **Depth:** 2 vs. 4 vs. 6 layers (loss decreased monotonically from 2.24 to 1.89 as depth increased).
2. **Heads:** 2 vs. 4 vs. 8 heads (4 heads proved optimal at d_model=128; 8 heads over-partitioned d_k to 16).
3. **Position:** Sinusoidal vs. Learned (Learned yielded marginal -0.003 lower loss but added parameters).

Figure 9: Ablation Study Comparison Charts

Figure 9: MiniGPT Controlled Ablation Studies



7. Packaging & Acceptance Tests

The model was packaged behind:

• **CLI:** python -m src.generate --prompt 'To be, or not to be' --max-new-tokens 80

• **FastAPI Service:** uvicorn src.api:app --port 8000 exposing /generate, /health, /info.

• **Unit Tests:** 10 passed tests verifying attention math, causal mask blocking, tensor shapes, and single-batch memorization.

8. Conclusion

A miniature 805k-parameter causal Transformer trained from scratch successfully learned the syntactic and phonetic structure of Shakespearean dialogue. All submission acceptance tests passed without error.